O que já foi feito e o que falta fazer

Resumo do backend de acompanhamento de obras, explicado para orientar a próxima etapa do grupo.

O produto que vocês estão construindo

A construtora deve cadastrar obras residenciais, organizar marcos, registrar avanços com fotos, vídeos ou documentos e publicar atualizações. Mudanças de prazo e estado precisam manter histórico; ocorrências devem registrar problemas e decisões.

Na documentação atual, a construtora usa **uma conta funcional**. Comprador, engenheiro e trabalhadores não fazem login. Exemplos de comprador e engenheiro são identificados como simulados. O prazo documentado do MVP é **03/11/2026**.

Estado real do repositório

Parte	Situação verificada
Documentação	10 documentos: visão, regras, funcionalidades, aceite, arquitetura, APIs externas, plano, casos de uso, testes e rastreabilidade.
Backend	Flask inicializado, conexão MongoDB configurada e três rotas de trabalhadores implementadas.
Testes	Cadastro de trabalhador e quatro cenários de campos faltantes passam. Cadastro de construtora falha antes da requisição.
Fluxo principal	Login, obras, marcos e publicação ainda não aparecem implementados neste checkout.

Como interpretar este resumo

Código implementado não significa fluxo completo aprovado. A revisão confirma o comportamento local e os testes com mocks; não comprova persistência no Atlas, uso pelo React nem implantação em EC2. O outro repositório do frontend não foi inspecionado.

Como as peças se conectam

Arquivo	Responsabilidade atual
api.py	Cria a aplicação Flask, habilita CORS e registra o blueprint de trabalhadores. Executa com debug quando iniciado diretamente.
db.py	Lê o .env, exige MONGO_URI, cria MongoClient, seleciona o banco da URI e a coleção tbl_trabalhadores.
trabalhadores.py	Define as rotas de cadastro, edição e exclusão; faz as operações na coleção e trata erros do MongoDB.
validacoes.py	Valida o JSON e os campos, além de normalizar textos, telefone, cargo e vínculos.
test_api.py	Usa pytest e mocks para exercitar o Flask sem acessar o banco real.
app/config.py e app/database.py	Existem, mas estão vazios. Ainda não substituem a configuração e conexão da raiz.
requirements.txt	Declara Flask, flask-cors, PyMongo, python-dotenv e pytest.

Caminho de uma requisição de cadastro

Cliente HTTP -> Flask -> rota de trabalhadores -> validação -> normalização -> MongoDB -> resposta JSON. O blueprint agrupa as rotas; não é outro servidor.

O que o cadastro salva

O cliente envia nome, email, telefone e cargo. A API define **obras** como uma lista com o código inteiro da URL, **ativo=True** e **data_cadastro** com horário UTC. O MongoDB gera o identificador e a resposta devolve seu valor como texto.

Isso é cadastro de uma pessoa vinculada à obra, **não criação de uma conta de acesso**. Não há senha nem sessão nesse fluxo. Cadastro de construtora também não está implementado.

Trabalhadores: o que funciona

Requisição	Comportamento
POST /trabalhadores/obra/{codigo}/	Cria trabalhador; exige código positivo e os quatro campos de texto. Retorna 201 com mensagem e ID.
PATCH /trabalhadores/{id}/	Atualiza somente campos enviados. Aceita nome, email, telefone, cargo e obras. Retorna 200; ID inválido retorna 400 e inexistente retorna 404.
DELETE /trabalhadores/{id}/	Exclui pelo ObjectId. Retorna 200 quando exclui, 400 para ID inválido e 404 se não existir.

Validações e normalização

O JSON precisa ser um objeto. Nome, email, telefone e cargo são textos obrigatórios no cadastro. O email passa por uma validação básica de formato; telefone aceita DDD com 10 ou 11 dígitos e determinados separadores.

São removidos espaços das extremidades dos textos. O telefone é armazenado só com números e o cargo em letras minúsculas. Na edição, obras deve ser uma lista de inteiros positivos; códigos repetidos são retirados e uma lista vazia remove os vínculos.

Dados inválidos retornam **422**. Na edição, JSON vazio e campos desconhecidos também são rejeitados. As rotas capturam erros do PyMongo e retornam **500** com uma mensagem genérica.

O que ainda falta neste módulo

Leitura/listagem de trabalhadores, autenticação, conferência de existência das obras e tratamento dos vínculos quando uma obra for excluída. Atualmente um código positivo pode ser salvo sem consultar se a obra existe.

Também não há verificação de email duplicado. A função trabalhador_para_dict prepara dados para consultas futuras, mas não existe rota GET que a utilize. A criação ainda não devolve o cabeçalho Location previsto no contrato geral.

O que os testes comprovam hoje

Execução verificada em 20/10/2026: 5 passaram e 1 falhou. Existem três funções de teste; uma delas é parametrizada em quatro execuções.

Teste	Resultado e significado
Cadastro de trabalhador	Passou. Confere 201, resposta JSON e dados enviados ao insert_one, incluindo obras, ativo e data_cadastro.
Campos faltantes	Quatro aprovações: remove nome, email, telefone ou cargo, um por execução; espera 422 e nenhuma inserção.
Cadastro de construtora	Falhou: o patch referencia usuarios.colecao_usuarios, mas o módulo usuarios não existe. A rota /usuarios também está ausente.

Por que usamos mocks

O cliente de teste executa o Flask real. O **patch** substitui a coleção usada pela rota por um objeto falso. Assim é possível definir o retorno do banco e verificar insert_one, update_one ou delete_one, sem salvar registros reais.

Antes de importar a API, o teste fornece uma MONGO_URI fictícia e simula MongoClient e load_dotenv. A fixture client prepara o cliente Flask; parametrize repete um teste com entradas diferentes. Não é necessário conftest.py para este arquivo único.

Correção pendente no teste de construtora

Enquanto usuarios.py não existir, retirar esse patch permite que o teste chegue à rota e falhe com 404, caso o grupo queira desenvolver começando pelo teste. Quando a coleção existir, o argumento do mock deve vir antes de client: **def test_cria_usuario_construtora(mock_colecao, client)**. Isso não substitui implementar a rota.

Git e execução

O histórico local mostra as branches correcao-teste e test_campos_faltantes integradas à main pelo PR #2. Na revisão, main estava alinhada à referência local origin/main e sem alterações pendentes. Isso não equivale a uma consulta nova ao remoto.

Para executar no PowerShell, na raiz do projeto:

```
.\.venv\Scripts\python.exe -m pytest -v
```

Mocks não comprovam persistência após reinício, integração com a interface ou serviços externos. Esses pontos exigem ambiente de teste e verificações de integração.

Começar pelos UC-01 a UC-04

A prioridade combinada na conversa é acesso e obras. Os testes de trabalhadores podem ser preservados; o próximo esforço deve preparar o caminho principal do produto.

Ordem	Implementar e verificar
1. UC-01 | CT-01 a 04	Conta pré-configurada com senha em hash; login, sessão, logout e proteção no servidor. Testar acesso válido, credenciais erradas, ausência/expiração de sessão e saída.
2. UC-02 | CT-05 a 10	Criar obra com dados obrigatórios, cliente e entrega por data exata OU intervalo. Validar datas e gerar autoria/data no servidor. Conferir persistência e falhas do banco.
3. UC-03 | CT-11 a 15	Listar, consultar e filtrar obras. Derivar atraso do prazo vencido; tratar lista vazia, filtros inválidos e ID inexistente.
4. UC-04 | CT-16 a 17	Editar apenas dados descritivos. Alterações de prazo e conclusão devem seguir o fluxo de mudança com motivo e histórico.

Esses são **17 casos documentados**, não necessariamente 17 funções Python. Vários contêm mais de um cenário e exigem interface ou banco real. O documento completo lista 111 casos; CT-107 a CT-110 dependem da integração externa escolhida e CT-111 verifica o ambiente publicado.

Depois dessa primeira trilha

Implementar marcos e dependências; avanços com mídia; edição de rascunho e publicação; correções versionadas; mudanças e histórico; ocorrências e aceitação sem solução; exclusões consistentes; conclusão do módulo de trabalhadores e exemplos simulados.

Escolher e validar a API externa com o professor, definir limites e acesso às mídias e preparar React -> Flask -> Atlas no EC2. CORS e debug precisam de configuração adequada ao ambiente publicado. A documentação de arquitetura ainda propõe rotas e detalhes de contrato que devem ser alinhados pelo grupo.

Regra para considerar uma entrega pronta

Para cada UC: combinar contrato -> escrever teste -> implementar -> executar com mocks -> verificar persistência e interface -> registrar evidência na issue/PR. Login de conta pré-configurada não depende de criar um cadastro público de construtora.

Fontes locais: README; api.py, db.py, trabalhadores.py, validacoes.py, test_api.py e requirements.txt; documentos docs/01 a docs/10. Os casos de uso/teste e as decisões atuais indicam uma conta funcional. Resultado limitado ao checkout revisado; não foram acessados Atlas, frontend ou EC2.